

On-Device Houseplant Species Classification

MobileNetV1 INT8 on the Arduino Nano 33 BLE Sense

Live on-device demo: the dashboard classifies an asparagus fern from the OV7675 camera feed.

Asaf J. Iron-Jobes

EE446: TinyML for Ultra Low-Power Edge Computing
Spring 2026



A six-class houseplant classifier that runs entirely on an Arduino microcontroller, with no cloud or network dependency.

2. Team Member Contributions

This is a solo project. I built the data collection pipeline, the data cleaning system, the training architecture and code, the compression pipeline, the firmware, and the host dashboard software. Python and TensorFlow were used throughout to keep full control over the design details and to practice an industry-relevant ML workflow. I performed all manual testing of the deployed system and designed the serial communication protocol between the Arduino and the host dashboard on my Mac. Claude Code was used as an AI coding assistant for parts of the implementation, debugging, and report drafting.

3. Introduction and Project Objective

The problem and why it suits TinyML. At face value, identifying a houseplant species is a low-stakes task. The motivation comes instead from how this problem is usually solved today. Most plant-identification mobile apps require a paid subscription and a live internet connection, sending each photo to a server for inference. The goal of this project is a TinyML alternative that runs entirely on a microcontroller, so it is free, fully offline, and low-power. These three properties are exactly what TinyML provides, and each one matters in a realistic deployment.

A more capable version of this classifier could support field research in remote areas such as the Amazon rainforest. In that setting the offline capability is the critical feature, because inference must work with no connectivity and on battery power for extended periods.

A closer-to-home application is a self-caring smart planter. It would use a planter with an embedded MCU that automatically identifies the species it was planted with, looks up its care profile (water, light, fertilizer) from an on-device database, and drives a simple actuation system to water and tend the plant. Here both the low-power and offline properties make the device essentially set and forget, with no app, account, or network setup required. This project is a proof-of-concept for that class of device: on-device, real-time houseplant classification with no cloud dependency.

Project goal, input and output, and target deployment. The system classifies a houseplant into one of six species chosen from plants available at home. The input is a live RGB image stream from an Arducam OV7675 camera. The output is the predicted species and a per-class confidence, produced entirely on-device. The target deployment platform is the Arduino Nano 33 BLE Sense (Nordic nRF52840, 256 KB RAM and 1 MB flash), with the model running under TensorFlow Lite for Microcontrollers and results streamed over serial to a live local dashboard. No network, server, or cloud inference is used at any point.

Success criteria. The project is considered successful if it meets three conditions. First, it runs the full capture, inference, and display loop on the Nano 33 BLE Sense in real time. Second, it fits within the device's memory budget, keeping the model and tensor arena within the 256 KB RAM and 1 MB flash limits. Third, it correctly identifies the six species on live camera frames at the demo, rather than only on held-out Python test data. Usability, meaning an immediate offline on-device prediction with no connectivity, is treated as a first-class goal alongside accuracy.

4. Dataset and Experimental Setup

Dataset source, sensor, classes, and collection. The task is a six-class image classification problem over six houseplant species: Aloe Vera, Asparagus Fern, Dumb Cane, Jade Plant, Money Tree, and Snake Plant. These species were chosen because all six were available at home, which made it possible to collect real photos and on-device camera data for each one. Three sources of data were used. The first is the public Kaggle “House Plant Species” image dataset, which provides the bulk of the training images (2,070 images across the six classes). The second is a set of phone photos that I took of my own plants (18 per class, 108 total), captured with a smartphone camera under typical indoor lighting. The third is camera data captured directly on the target hardware using the Arducam OV7675 module on the Arduino Nano 33 BLE Sense (100 images per class, 600 total). The OV7675 is a 0.3 MP CMOS sensor, and these frames are representative of what the model actually sees at deployment time: low resolution, indoor lighting, and short working distance.

Preprocessing, normalization, and augmentation. All images are converted to RGB and resized to a fixed 96 by 96 input. Because the phone photos carried EXIF orientation metadata that the training pipeline does not honor automatically, each image is passed through an EXIF transpose step during preparation so that it is stored upright; without this, several classes were fed in sideways and the model trained near chance. Files in mixed or hidden formats (WebP and MPO disguised as .jpg) are normalized to standard JPEG during the same pass. Pixel values are scaled into the range expected by the MobileNet backbone. Training-time augmentation includes standard geometric and photometric transforms (random flips, rotation, zoom, and brightness or contrast jitter). For the on-device fine-tuning stage, an additional “degrade” augmentation is applied probabilistically to roughly half of the samples to simulate OV7675 sensor characteristics (lower effective resolution, noise, and color shift), so the general-data model is pushed toward the look of the real camera. Labels are taken directly from the class folder names and encoded as integer class indices.

Train, validation, and test splits and leakage avoidance. The main dataset is split per class into 70 percent train, 15 percent validation, and 15 percent test (1,521 / 324 / 333). The Arduino camera dataset is split per class into 80 percent train, 10 percent validation, and 10 percent test (480 / 60 / 60). All splitting is done at the file level with a fixed random seed so the splits are reproducible, and the source folder of original images is never modified during preparation; the pipeline only ever copies out of it. To avoid data leakage, the splits are disjoint by file, augmentation is applied only after splitting (so no augmented copy of a test image can appear in training), and the validation and test sets are held out and never used for gradient updates. The one honest limitation is in the Arduino set: the 100 images per class were captured in the same sessions, so the camera test frames are correlated with the camera training frames. The camera test accuracy should therefore be read as optimistic, and the live demo is treated as the true measure of real-world performance.

Onboard-sensor data: collection and use. The Arduino camera data was collected through a dedicated capture pipeline that I wrote for this project. The Nano 33 BLE Sense runs a capture sketch that drives the OV7675, and on a request from a host-side Python script it returns one raw RGB565 frame over serial, framed with a magic-byte header for synchronization. The host script

decodes the frame and saves it as an image, letting me build a labeled on-device dataset one species at a time. This onboard data is used in two ways. It serves as a dedicated camera test set to measure how the model performs on real sensor frames rather than only on clean Kaggle images, and it drives a second, environment-specific fine-tuning stage in which the model trained on the general data is adapted to the OV7675 domain. This directly satisfies the advanced component of fine-tuning a model using data collected from the target hardware, and the comparison of general-data versus camera-fine-tuned performance is reported in Section 7.

5. Model Development and Advanced Components

Big ML model trained offline. The core model is a convolutional neural network built on a MobileNetV1 backbone (width multiplier $\alpha = 0.25$) pretrained on ImageNet, with a custom classification head (global average pooling, dropout 0.3, and a six-way softmax dense layer). The input is 96x96 RGB. This single architecture is trained in three offline stages: a head-only stage on the general dataset, a backbone fine-tune stage, and a camera (“arducam”) domain-specialization stage. The fully trained float Keras model is the “big” model, reaching 81.7 percent accuracy on the camera test set and 69.4 percent on the general (Kaggle plus phone) test set. MobileNetV2 and V3 were considered as alternatives but rejected before training for memory reasons (see Section 8), so they are compared by analysis rather than by a separately trained model.

TinyML-ready model. The deployed model is a full-integer INT8 version of the trained network, produced by post-training quantization (PTQ) in TensorFlow Lite. All weights and activations are quantized to int8, and the input and output tensors are kept int8 to match the camera and the microcontroller. Calibration uses a representative dataset of camera frames so the quantization ranges match the deployment camera. This compresses the model from 849 KB (float) to 303 KB (int8), a 2.8x reduction, while preserving camera test accuracy exactly at 81.7 percent and slightly improving general test accuracy (69.4 to 71.5 percent). The int8 model is exported to a C header and run on-device under TensorFlow Lite for Microcontrollers.

Advanced components. Only one advanced component is required for a solo team; this project includes three. First, fine-tuning on onboard-sensor data: the model is fine-tuned on images captured through the OV7675 camera on the target board, adapting it from clean internet photos to the real deployment domain. This stage uses layer freezing, a low learning rate, early stopping on the camera validation set, and rehearsal (mixing a fraction of general data into each batch) to specialize without forgetting the general features. Second, a communication method beyond the Serial Monitor: inference results are streamed over serial to a live local dashboard that shows the predicted species, per-class confidence bars, and a reconstructed view of the model's input frame. Third, post-training quantization as a model-compression technique.

Training details. The loss is sparse categorical cross-entropy and the optimizer is Adam throughout. The schedule is: Stage 1 (head only, backbone frozen) at learning rate $1e-3$ for up to 18 epochs; Stage 2 (top 30 backbone layers unfrozen) at $1e-5$ for up to 10 epochs; Stage 3 (camera specialization, top 20 layers unfrozen, rehearsal weight 0.25) at $1e-4$ for up to 30 epochs. The batch size is 32. Early stopping is used in every stage (patience 4 on validation loss for the main stages, patience 6 on camera validation accuracy for specialization, restoring the best

weights). Class weighting handles the imbalance in the general dataset. Software: TensorFlow and Keras for training, TensorFlow Lite for quantization and conversion, with NumPy and Pillow for data handling.

Why the deployed model was selected. The deployed INT8 MobileNetV1 was chosen over the float model and the V2/V3 alternatives mainly for fit on the hardware. MobileNetV1 fits within the 256 KB RAM budget while V2 and V3 do not (the memory analysis is in Section 8). INT8 was chosen over the float model because PTQ shrank the model 2.8x with no loss in camera accuracy, making it both smaller in flash and faster while remaining accurate on the deployment domain. The result is the smallest, fastest variant that meets the accuracy goal and fits the device's memory budget.

6. TinyML System Design and On-Device Deployment

System pipeline. Each frame flows through a fixed pipeline on the Arduino. The OV7675 camera captures a QCIF (176x144) RGB565 frame. The frame is center-cropped to a square and downsampled to 96x96, then each pixel is quantized to int8 using the model's input scale and zero point. The INT8 MobileNetV1 runs under TensorFlow Lite for Microcontrollers and produces six softmax scores. A 5-frame peak-hold plus a 0.60 confidence threshold smooth the output and suppress low-confidence frames, after which the result and a downsampled preview of the model's input are streamed over serial to a local dashboard. Figure 1 shows the full path from sensor to display.

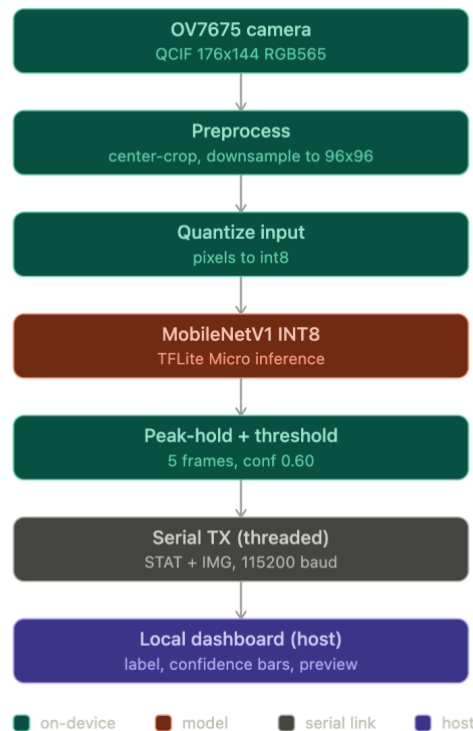


Figure 1. Full on-device pipeline from OV7675 capture to the host dashboard.

Model conversion for deployment. The trained Keras model is converted to a full-integer INT8 TensorFlow Lite model with post-training quantization, using a representative dataset of camera frames for calibration and keeping the input and output tensors int8. The resulting .tflite file is turned into a C source array with `xxd -i` and stored as `model_int8.h`. The array is marked `alignas(16) const` so it is placed in flash rather than RAM (an early bug placed it in RAM and broke the link). On the Arduino, the model is loaded with `tflite::GetModel`, and the needed operators are registered explicitly through a `MicroMutableOpResolver` holding eight ops (Conv2D, DepthwiseConv2D, FullyConnected, Add, Mul, Mean, Pad, and Softmax) rather than the full op set, which keeps the firmware small. A `MicroInterpreter` then calls `AllocateTensors` over a statically reserved tensor arena, and `Invoke` runs inference each frame. The build uses the `Arduino_TensorFlowLite` library on the Arduino Mbed OS Nano core.

Deployment constraints. The deployed INT8 model is 303 KB in flash (down from 849 KB float). The tensor arena reserves 112 KB and uses 100.4 KB as measured on-device by `arena_used_bytes()` at boot. This is the dominant RAM cost and fits within the Nano 33 BLE Sense's 256 KB of RAM, leaving room for the camera frame buffer, the preview buffer, and the TX thread's stack. MobileNetV1 was required because V2 and V3 exceed the RAM limit at 96x96 input (see Section 8). Host-side inference latency is about 0.15 ms per frame for the INT8 model versus 0.28 ms for the float model, confirming the INT8 model is both smaller and faster; true on-device latency is bounded in practice by camera read and serial TX rather than by `Invoke`.

On-device inference evidence. On-device results are shown through a live local dashboard rather than only the Serial Monitor, as seen on the cover page. The full demo video (far more than ten examples) is available at the link below, and a representative set of ten instances is recorded in Table 5.

[Demo videos link](#)

Communication method and data format. Inference results are transmitted over USB serial at 115200 baud in a simple two-message protocol consumed by a Python (matplotlib) dashboard. A STAT line carries the smoothed result and the six per-class confidences as text: STAT <idx> <conf> <c0..c5>, where idx is the peak-hold class index or -1 when below threshold. An IMG message carries the model's-eye preview: a header IMG 96 96, a 4-byte magic marker 0x55AA55AA for resynchronization, then 96x96 RGB565 pixels. The dashboard parses these to display the predicted species and confidence, with a green bar on the detected class, and renders the preview image so the operator sees exactly what the model sees. To keep the interface smooth, the firmware uses two Mbed RTOS threads: the main thread captures and runs inference while a dedicated TX thread streams serial, with two semaphores pipelining them so transmission of one frame overlaps capture and inference of the next.

7. Results and Analysis

Evaluation metrics. Accuracy and macro-averaged F1 score are reported for all conditions. The six-class distribution is balanced across the main test set and exactly balanced across the camera test set, so accuracy is a meaningful primary metric. Macro-F1 is included because it weights

each class equally regardless of count and catches any per-class imbalance in precision or recall that accuracy alone could mask.

Big-model vs. TinyML comparison

Metric	Float model	INT8 model (deployed)
Camera test accuracy	81.67%	81.67%
Camera macro-F1	0.816	0.814
General test accuracy	69.37%	71.47%
General macro-F1	0.690	0.715
Flash size	849 KB	303 KB
Size reduction vs. float	—	2.8x smaller
Tensor arena (used / reserved)	n/a	100.4 KB / 112 KB
Host inference latency (estimate)	~0.28 ms	~0.15 ms

Table 1. Float vs. INT8 comparison. Host latency is a Python-side estimate; on-device latency is dominated by camera capture and serial TX, not by model Invoke.

Per-class metrics: deployed INT8 model

Class	Precision	Recall	F1	Support
Aloe Vera	0.727	0.800	0.762	10
Asparagus Fern	0.769	1.000	0.870	10
Dumb Cane	0.900	0.900	0.900	10
Jade Plant	0.778	0.700	0.737	10
Money Tree	0.750	0.600	0.667	10
Snake Plant	1.000	0.900	0.947	10
Macro average	0.821	0.817	0.814	60

Table 2. INT8 (deployed) model, camera test set.

Class	Precision	Recall	F1	Support
Aloe Vera	0.762	0.780	0.771	41
Asparagus Fern	0.657	0.793	0.719	29
Dumb Cane	0.719	0.753	0.736	85
Jade Plant	0.717	0.579	0.641	57
Money Tree	0.717	0.741	0.729	58
Snake Plant	0.705	0.683	0.694	63
Macro average	0.713	0.722	0.715	333

Table 3. INT8 (deployed) model, general test set.

Per-class metrics: float reference model

Class	Precision	Recall	F1	Support
Aloe Vera	0.727	0.800	0.762	10
Asparagus Fern	0.900	0.900	0.900	10
Dumb Cane	0.750	0.900	0.818	10
Jade Plant	0.778	0.700	0.737	10
Money Tree	0.875	0.700	0.778	10
Snake Plant	0.900	0.900	0.900	10
Macro average	0.822	0.817	0.816	60

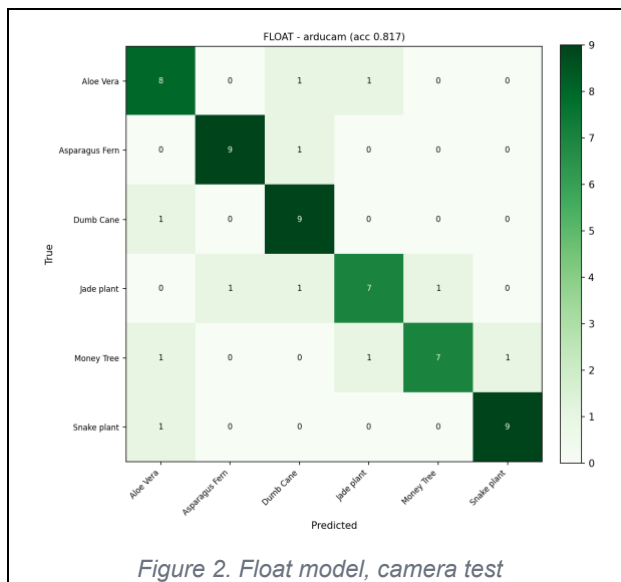
Table 4a. Float model, camera test set.

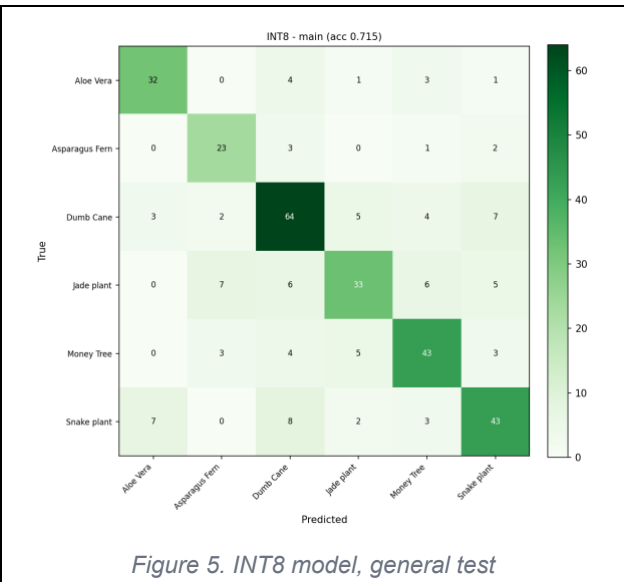
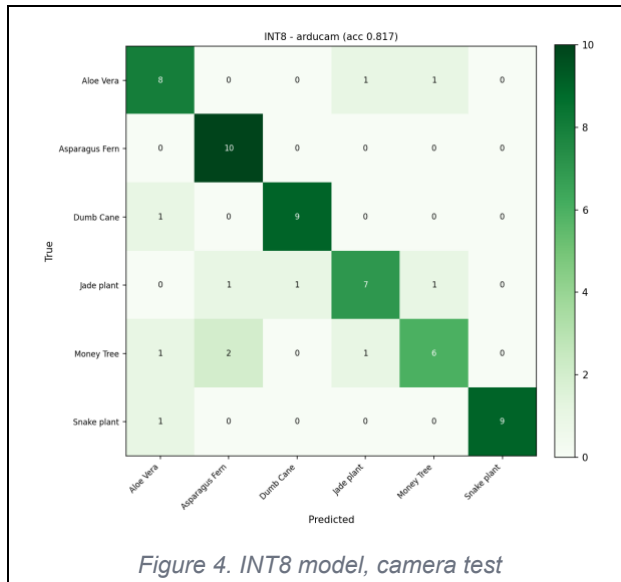
Class	Precision	Recall	F1	Support
Aloe Vera	0.721	0.756	0.738	41
Asparagus Fern	0.521	0.862	0.649	29
Dumb Cane	0.747	0.694	0.720	85
Jade Plant	0.721	0.544	0.620	57
Money Tree	0.729	0.741	0.735	58
Snake Plant	0.689	0.667	0.677	63
Macro average	0.688	0.711	0.690	333

Table 4b. Float model, general test set.

Confusion matrices

Figures 2 through 5 show the confusion matrices (rows are the true class, columns the predicted class) for the four conditions.





Deployment resource summary

Resource	Float model	INT8 model (deployed)
Flash (model size)	849 KB	303 KB
Tensor arena reserved	n/a	112 KB
Tensor arena used (on-device)	n/a	100.4 KB
Host inference latency (estimate)	~0.28 ms	~0.15 ms
On-device latency	n/a (not deployed)	bounded by camera + serial TX

Table 5. Deployment resource summary on the Arduino Nano 33 BLE Sense. Host latency is a Python-side estimate; on-device latency is not directly measured.

On-device inference evidence: View the videos on [GitHub](#) for deployed model accuracy demo

Analysis and key findings

The INT8 model matches the float model on camera accuracy (81.67 percent on both) and slightly outperforms it on the general test set (71.47 percent vs. 69.37 percent), confirming that full-integer quantization introduces no meaningful accuracy degradation for this task and can even act as a mild regularizer. The 2.8x flash reduction (849 KB to 303 KB) is a direct result of INT8 post-training quantization and is the primary reason the model fits comfortably within the 1 MB flash budget. The camera-specialized fine-tuning stage accounts for the gap between general-set and camera-set accuracy: the camera accuracy is higher because the model was specifically adapted to OV7675 image characteristics. In the per-class results, the classes with the lowest recall (Money Tree and Jade Plant) are visually similar succulents, and the confusion matrices show they are most often confused with each other and with Aloe Vera.

8. Challenges, Limitations, and Future Work

Challenges. Two design decisions were forced by the memory budget of the Arduino Nano 33 BLE Sense. The first is the 96x96 input resolution. The 256 KB RAM ceiling prevents using a larger input, which means the model sees a coarser, lower-detail view of each plant than a phone camera would provide. Training on clean, high-resolution Kaggle images under this constraint produced a model that generalized poorly to the blurry, noisy OV7675 sensor output. The solution was to collect dedicated on-device camera data and add a domain-specialization fine-tuning stage; without it, camera test accuracy was significantly lower than the general-data accuracy, and the model would have failed on live hardware even while appearing to perform well on held-out Python test data. The second challenge was the backbone architecture choice. MobileNetV2 and V3 were the natural first choices because they are more accurate than V1 at equivalent width, but their inverted-residual expansion blocks push peak activation memory to roughly 293 KB at 96x96, which exceeds the 256 KB limit. Switching to MobileNetV1, whose activations measure roughly 100 KB at the same resolution, resolved the memory constraint.

Limitations. Three limitations constrain how confidently the reported results generalize. First, the camera test set is correlated with the camera training set: all 600 on-device images were captured in the same sessions and the same physical environment, so the 81.67 percent camera test accuracy is optimistic relative to a truly independent test, and the live demo is the honest measure of real-world performance. Second, the custom dataset is small: 18 phone images and 100 on-device images per class. A larger and more diverse collection of real-sensor images (different lighting, backgrounds, and plant ages) would improve robustness. Third, the classifier covers only six species, so plants outside these six classes receive a forced prediction within the known set, which can be misleading in a real deployment.

Future work. Several extensions would strengthen the system. Expanding to more species is the most direct improvement; the pipeline is parameterized so adding classes requires only new data collection and retraining. Adding on-device fine-tuning would let the model adapt to a new environment or new species without a PC, which is important for the self-caring planter use case. Measuring true on-device inference latency with microsecond-resolution timers around `Invoke` would replace the current host-side estimate with a hardware-accurate number. Finally, adding battery-powered operation and measuring current draw during inference would let the system report an energy cost per prediction, a core TinyML metric for real field deployment.

9. Reproducibility and References

The full implementation is available in the project [GitHub repository](#). The Python pipeline is organized as single-purpose scripts driven by a central `config.py` that sets all paths, image size, split ratios, learning rates, and the fixed random seed: `prepare_data.py` and `prepare_arduino.py` build the train, validation, and test splits; `train.py` runs the three-stage training; `quantize.py` performs INT8 post-training quantization and TFLite conversion; and `report_metrics.py` generates the results report. On-device, `plant_infer.ino` runs inference and `dashboard.py` is the host dashboard. The training environment is Python 3.11, TensorFlow 2.14, and Keras 2.14, and deployment uses the `Arduino_TensorFlowLite` library on the Mbed OS Nano core. A fixed seed (42) makes the data splits and training reproducible across runs.

References

- [1] “House Plant Species” image dataset, Kaggle.
- [2] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- [3] M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR 2018, arXiv:1801.04381.
- [4] J. Deng et al., “ImageNet: A Large-Scale Hierarchical Image Database,” CVPR 2009 (pretrained backbone weights).
- [5] TensorFlow and Keras, model training framework, <https://www.tensorflow.org>.
- [6] TensorFlow Lite and TensorFlow Lite for Microcontrollers, <https://www.tensorflow.org/lite/microcontrollers>.
- [7] Arduino_TensorFlowLite library and Arduino Mbed OS Nano core, Arduino, <https://www.arduino.cc>.
- [8] Harvard TinyMLx, TinyMLShield and Arduino_OV767X camera examples, <https://github.com/tinyMLx> (camera capture and TFLM examples adapted from EE446 Lab 5).
- [9] NumPy, Pillow, pySerial, and Matplotlib, Python libraries.
- [10] Anthropic, Claude Code (Opus 4.x), AI coding assistant, 2025, <https://claude.com/claude-code> (used for pipeline development, debugging, and report drafting).